PRECONDITIONING ADMM FOR FAST DECENTRALIZED OPTIMIZATION

Meng Ma *Georgios B. Giannakis*

Digital Technology Center and Dept. of ECE
University of Minnesota
Email: {maxxx971, georgios}@umn.edu

ABSTRACT

In this work, we consider the distributed optimization problem using networked computing machines. Specifically, we are interested in solving this problem using the alternating direction method of multipliers (ADMM) while accounting for edge weights. Existing works focus on *star* graphs and use simple *heuristics* for other types of graphs. The present work shows that the optimal edge weights design is equivalent to the preconditioning matrix of ADMM that leads to the fastest convergence speed. Based on a tight convergence rate of ADMM, we show that the preconditioning matrix of general graphs can be found by minimizing the ratio of the largest and smallest nonzero eigenvalue of the graph Laplacian. Numerical experiments show that preconditioned ADMM converges much faster to a certain accuracy with less communication rounds, and exemplify the robustness to topology changes of the underlying network.

Index Terms— Decentralized optimization, ADMM, preconditioning, hybrid ADMM

1. INTRODUCTION

Consider the distributed optimization problem using N networked computing machines

$$\min_{\mathbf{x}} \sum_{i=1}^{N} f_i(\mathbf{x}) \tag{1}$$

where f_i is the local objective function only accessible to node i and $\mathbf{x} \in \mathbb{R}^l$ is the common decision variable. Distributed optimization problems arise when the computation or storage load is beyond the capability of a single machine, or data are collected at dispersed locations. Examples includes multi-agent coordination, large scale machine learning, distributed signal processing over sensor networks [1–4].

Problem (1) is typically formulated in consensus form [1, 2, 5–7]. ADMM is particularly appealing for such problems thanks to the mild condition for convergence and its ability to decompose the objective into easier subproblems [2, 8]. Typically, ADMM-based algorithms alternate between a variables update step and a communication step. Based on the communication pattern, these algorithms can be further classified as centralized ADMM (CADMM) [2, 8] or decentralized ADMM (DADMM) [9, 10]. The applicability of CADMM is quite limited due to its reliance on the star topology. Though DADMM can be used for any kind of graphs, it cannot adapt to the specific network topology.

The hybrid ADMM (HADMM) takes node importance into consideration by locally aggregating updates [11]. This paper considers the weighted hybrid ADMM (WHADMM), which takes into account both node importance and edge weights. We show that designing edge weights is equivalent to preconditioning ADMM. There are some works that considers improving performance of ADMM via preconditioning [12], but such results only apply to problems defined over star graphs. They also suggest to use the same technique as a heuristic for other graphs, and there is no guarantee that this will work. Another work considers the optimal edge weights while solving problem (1) using DADMM [13]. Based the linear convergence rate, they propose to maximize this rate by minimizing some key terms. Our approach not only works for any graph, but also naturally applies to DADMM upon properly constructing the communication graph.

2. HYBRID ADMM OVER WEIGHTED GRAPHS

The decentralized optimization problem aims to solve (1) using N networked nodes. The common variable $\mathbf{x}$ prevents parallel processing. To decouple the problem, each machine maintains its own local variable, and neighbors must agree on common values, which translates to formulation

$$\begin{aligned} \min_{\{\mathbf{x}_i\}} \quad & \sum_{i=1}^{N} f_i(\mathbf{x}_i) \\ \text{s. to} \quad & \mathbf{x}_i = \mathbf{w}_{ij}, \mathbf{x}_j = \mathbf{w}_{ji}, \quad i,j \text{ connected} \end{aligned} \tag{2}$$

where $\mathbf{w}_{ij} = \mathbf{w}_{ji}$ are auxiliary variables introduced for each connected pair of nodes (i, j). Problem (2) is amenable to ADMM, which allows parallel minimization of each local cost function. A valid solution can be obtained by periodically exchanging information among neighbors and finally all machines reach a consensus.

This work is supported by NSF grant 1901134.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 06,2025 at 09:34:47 UTC from IEEE Xplore. Restrictions apply.

The hybrid ADMM [11] generalizes the pairwise connections in (2), which could be modeled by a *hypergraph* $\mathcal{H} := (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} := \{1, 2, \ldots, N\}$ collects all nodes, and the edge set $\mathcal{E}_{\mathcal{H}} := \{\mathcal{E}_1, \ldots, \mathcal{E}_M\}$ collects all *hyperedges*. Each hyperedge is a set of connected nodes. The idea of *hyperedge* is a generalization of *pairwise connection*, and nodes residing in the same hyperedge can exchange information, thus becoming *neighbors*. If the hypergraph is connected, then $\bigcup_{j=1}^{M} \mathcal{E}_j = \mathcal{V}$, where $|\mathcal{E}_{\mathcal{H}}| = M$. Then the consensus constraint is mathematically equivalent to $\mathbf{x}_i = \mathbf{z}_j,\ i \in \mathcal{E}_j$ where $\mathbf{z}_j$ is the auxiliary variable assigned to hyperedge $\mathcal{E}_j$.

Writing the constraints in matrix form reveals the intrinsic structure of this problem.Specifically, we can write all constraints $\mathbf{x}_i = \mathbf{z}_j,\ i \in \mathcal{E}_j$ into one linear equation $\mathbf{A}\mathbf{x} = \mathbf{B}\mathbf{z}$, where $\mathbf{x} \in \mathbb{R}^{Nl}$ is the vector obtained by concatenating all $\mathbf{x}_i \in \mathbb{R}^l$ and $\mathbf{z} \in \mathbb{R}^{Ml}$ by concatenating all $\mathbf{z}_j \in \mathbb{R}^l$. The rows of matrices $\mathbf{A}$ and $\mathbf{B}$ essentially select the corresponding $\mathbf{x}_i$ and $\mathbf{z}_j$ for each constraint. Let T denote the total number of constraints and label all constraints in arbitrary order. Then $\mathbf{A} \in \mathbb{R}^{Tl \times Nl}$ and $\mathbf{B} \in \mathbb{R}^{Tl \times Ml}$ are defined such that if the t-th constraint is $\mathbf{x}_i = \mathbf{z}_j$, then (t, i)-th block of $\mathbf{A}$ and (t, j)-th block of $\mathbf{B}$ are identity matrices; other elements are zeros.

Another important matrix is the (signless) incidence matrix $\mathbf{C} \in \mathbb{R}^{Nl \times Ml}$, whose (i, j)-th block is $\mathbf{I}_l$ if i-th node belongs to j-hyperedge, and $\mathbf{0}_l$ otherwise. The degree of a node counts the number of connected edges, while the degree of an hyperedge counts the number of connected nodes. Define block diagonal matrices $\mathbf{D} := \operatorname{diag}(d_1, \ldots, d_N) \otimes \mathbf{I}_l$, $\mathbf{E} := \operatorname{diag}(e_1, \ldots, e_M) \otimes \mathbf{I}_l$, where d_i is the degree of node i and e_j the degree of hyperedge j. Then the following relations hold: $\mathbf{A}^\top\mathbf{A} = \mathbf{D}, \mathbf{B}^\top\mathbf{B} = \mathbf{E}, \mathbf{A}^\top\mathbf{B} = \mathbf{C}$ ([11, Lemma 1]).

Example 1. Consider a decentralized optimization problem over a graph shown in Fig. 1(a) with $l = 1$. There are 4 nodes and 2 hyperedges, namely $\mathcal{E}_1 := \{1, 2\}$ and $\mathcal{E}_2 := \{2, 3, 4, 5\}$, shown as gray dashed circles. Clearly, this grouping scheme results in a connected hypergraph. Based on the current grouping, the matrices related to this hypergraph are given by

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \mathbf{C} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}.$$

One can verify that the relations among important matrices indeed hold.

It is often necessary to take into consideration the relative importance of edges while designing algorithms for decentralized optimization. It turns out that weight information can be represented by a diagonal scaling matrix left multiplied to linear constraint $\mathbf{A}\mathbf{x} = \mathbf{B}\mathbf{z}$. Let $\mathbf{S} \in \mathbb{R}^{Tl \times Tl}$ be the scaling matrix such that $\mathbf{S}^2 = \operatorname{diag}(w_1, w_2, \ldots, w_T) \otimes \mathbf{I}_l$, where w_t is the weight associated with t-th constraint. Accounting for edge weight information, problem (1) can be formulated as

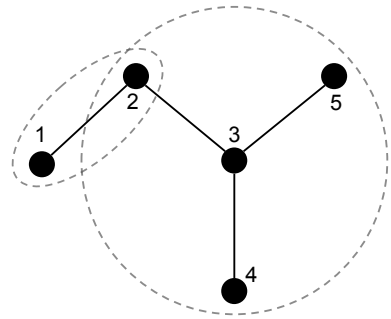


Fig. 1: An example graph.

$$\min_{\{\mathbf{x}_i, \mathbf{z}_j\}} \quad \sum_{i=1}^{n} f_i(\mathbf{x}_i) \quad \text{subject to} \quad \mathbf{S}\mathbf{A}\mathbf{x} = \mathbf{S}\mathbf{B}\mathbf{z}. \tag{3}$$

Matrices $\hat{\mathbf{A}} = \mathbf{S}\mathbf{A}, \hat{\mathbf{B}} = \mathbf{S}\mathbf{B}$ can be viewed as the weighted counterparts of $\mathbf{A}$ and $\mathbf{B}$, and matrices $\hat{\mathbf{C}}$, $\hat{\mathbf{D}}$, and $\hat{\mathbf{E}}$ can be defined consequently.

2.1. Algorithm

Problem (3) is ready to be solved using ADMM. Let $\boldsymbol{\nu}_t \in \mathbb{R}^l$ be the Lagrange multiplier of the t-th constraint, and $\boldsymbol{\nu} = (\boldsymbol{\nu}_1^\top, \ldots, \boldsymbol{\nu}_T^\top)^\top \in \mathbb{R}^{Tl}$. At iteration k, ADMM updates for solving (3) reduce to

$$\begin{aligned} \mathbf{x}^{k+1} &= \operatorname*{argmin}_{\mathbf{x}} f(\mathbf{x}) + \frac{\rho}{2}\mathbf{x}^\top\hat{\mathbf{D}}\mathbf{x} + \langle \mathbf{x}, \mathbf{y}^k - \gamma\hat{\mathbf{C}}\hat{\mathbf{E}}^{-1}\hat{\mathbf{C}}^\top\mathbf{x}^k \rangle \\ \mathbf{y}^{k+1} &= \mathbf{y}^k + \gamma(\hat{\mathbf{D}} - \hat{\mathbf{C}}\hat{\mathbf{E}}^{-1}\hat{\mathbf{C}}^\top)\mathbf{x}^{k+1} \end{aligned} \tag{4}$$

where $\mathbf{y}^k = \mathbf{A}^T\boldsymbol{\nu}^k$ is a change of variable, γ a penalty parameter. Variable $\mathbf{z}$ has been eliminated [11].

Updates (4) are fully separable across nodes and edges, thanks to the structures of $\hat{\mathbf{D}}, \hat{\mathbf{E}}$ and $\hat{\mathbf{C}}$. Towards this end, notice that $\hat{\mathbf{D}}$ is block diagonal, thus both $\hat{\mathbf{D}}\mathbf{x}$ and $\mathbf{x}^\top\hat{\mathbf{D}}\mathbf{x}$ are fully separable across nodes. It remains to show $\hat{\mathbf{C}}\hat{\mathbf{E}}^{-1}\hat{\mathbf{C}}^\top\mathbf{x}$ is also separable across nodes. Without loss of generality, let $l = 1$. Define $\mathbf{u} = \hat{\mathbf{C}}^\top\mathbf{x}$, then the i-th block of $\mathbf{u}$ can be obtained via $\mathbf{u}_j = \sum_{i=1}^{n} \hat{c}_{ij}\mathbf{x}_i = \sum_{i \in \mathcal{E}_j} \hat{c}_{ij}\mathbf{x}_i$. Consequently, let $\mathbf{v} = \hat{\mathbf{C}}\mathbf{z} = \hat{\mathbf{C}}\hat{\mathbf{E}}^{-1}\hat{\mathbf{C}}^\top\mathbf{x}$, i-th block of $\mathbf{v}$ can be obtained by $\mathbf{v}_i = \sum_{j=1}^{M} \hat{c}_{ij}\mathbf{z}_j = \sum_{i \in \mathcal{E}_j} \hat{c}_{ij}\mathbf{z}_j$, which can be computed via neighborhood communication. It follows that per node updates can be implemented as

$$\begin{aligned} \mathbf{x}_i^{k+1} &= \operatorname*{argmin}_{\mathbf{x}_i} f_i(\mathbf{x}_i) + \frac{\rho}{2}d_i\|\mathbf{x}_i\|^2 + \mathbf{x}_i^\top(\mathbf{y}_i^k - \rho\mathbf{v}_i^k) \\ \mathbf{y}_i^{k+1} &= \mathbf{y}_i^k + \rho(d_i\mathbf{x}_i^{k+1} - \mathbf{v}_i^{k+1}). \end{aligned} \tag{5}$$

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 06,2025 at 09:34:47 UTC from IEEE Xplore. Restrictions apply.

Remark. The matrix $\hat{\mathbf{L}_{\mathcal{G}}} = \hat{\mathbf{D}} - \hat{\mathbf{C}}\hat{\mathbf{E}}^{-1}\hat{\mathbf{C}}^\top$ is a valid graph Laplacian, i.e., $\hat{\mathbf{L}_{\mathcal{G}}}\mathbf{1} = \mathbf{0}$ [11]. In fact, it is the Laplacian of the hypergraph that models the connectivity of nodes [14, Definition 2.1].

2.2. Convergence Analysis

The linear convergence rate can be established depending on the row rank of $\mathbf{A}$. In both cases, we assume the objective $f = \sum_i f_i$ is strongly convex and Lipschitz continous, which is precisely characerized in Assumption 1.

Assumption 1. *Both $f - \frac{1}{2}\|\cdot\|^2_{\mathbf{M}}$ and $\frac{1}{2}\|\cdot\|^2_{\mathbf{H}} - f$ are convex, where $\mathbf{H}$ and $\mathbf{M}$ are positive definite and $\alpha = \lambda_{min}(\mathbf{M})$, $\beta = \lambda_{max}(\mathbf{H})$.*

Proposition 1. *If $\mathbf{A}$ has full row rank, then the sequence $\{\boldsymbol{\nu}^k\}$ converges linearly, i.e., $\|\boldsymbol{\nu}^k - \boldsymbol{\nu}^\star\| \leq \left(\frac{\sqrt{\kappa}}{\sqrt{\kappa}+1}\right)^k C$, for some constant C and*

$$\kappa = \frac{\lambda_{max}(\mathbf{SAH}^{-1}\mathbf{A}^\top\mathbf{S}^\top)}{\lambda_{min}(\mathbf{SAM}^{-1}\mathbf{A}^\top\mathbf{S}^\top)}. \tag{6}$$

This rate is tight in the sense that there exist a certain class of problems satisfying the assumption can actually achieve this rate of convergence, see [12] for proof and examples.

Proposition 2 ([15]). *If $\mathbf{A}$ is not full row rank, and $\gamma = (\alpha\beta\lambda_{max}(\hat{\mathbf{L}_{\mathcal{G}}}))/(\|\mathbf{A}\|2(\lambda_{min>0}(\hat{\mathbf{L}_{\mathcal{G}}}))^2)$, then the sequence $\{\boldsymbol{\nu}^k\}$ converges with linearly, that is, $\|\boldsymbol{\nu}^k - \boldsymbol{\nu}^\star\| \leq \left(\frac{\sqrt{\kappa_d}}{\sqrt{\kappa_d}+1}\right)^k C$, for some constant C and*

$$\kappa_d = \frac{\beta}{\alpha}\frac{\|\mathbf{A}\|^2\lambda_{max}(\hat{\mathbf{L}_{\mathcal{G}}})}{(\lambda_{min>0}(\hat{\mathbf{L}_{\mathcal{G}}}))^2}. \tag{7}$$

Interestingly, when $\mathbf{A}$ has full row rank, the rate (7) reduces to that of (6) [15].

3. PRECONDITIONING ADMM

This section introduces two preconditioning techniques that aims to improve the convergence rate of hybrid ADMM through properly selecting diagonal scaling matrix, which ultimately translates to edge weights. Without loss of generality, we only consider the case $l = 1$ for notational simplicity.

3.1. Case I: A has full row rank

When $\mathbf{A}$ has full row rank, Proposition 1 characterizes the dependence of linear convergence rate on diagonal scaling matrix $\mathbf{S}$. To improve the convergence speed, one can simply find a proper $\mathbf{S}$ that yields the smallest κ (6), which naturally leads to an optimization problem $\min_{\mathbf{S}\in\mathcal{S}} \kappa$, where $\mathcal{S}$ denotes the set of all $T \times T$ positive definite diagonal matrices. This ratio arises in a lot of control problems, see e.g. [16]. This optimization problem can be expressed as an generalized eigenvalue problem and solved efficiently, see e.g. [17, 18].

3.2. Case II: A is row rank deficient

When $\mathbf{A}$ does not have full row rank, the convergence result Proposition 1 does not hold anymore. In this case, the diagonal scaling technique fails as $\mathbf{AM}^{-1}\mathbf{A}^\top$ is rank deficient and $\lambda_{\min}(\mathbf{AM}^{-1}\mathbf{A}^\top) = 0$. We introduce two approaches to circumvent this issue.

Approach I. Due to the rank deficiency, one could use the smallest nonzero eigenvalue, and consider instead

$$\min_{\mathbf{S}\in\mathcal{S}} \quad \frac{\lambda_{\max}(\mathbf{AH}^{-1}\mathbf{A}^\top)}{\lambda_{\min>0}(\mathbf{AM}^{-1}\mathbf{A}^\top)} \tag{8}$$

where $\lambda_{\min>0}(\cdot)$ is the smallest nonzero eigenvalue. Algorithms for solving this problem can be found in [17, 19].

Approach II. For the case $\mathbf{A}$ does not have full row rank, Proposition 2 provides an alternative convergence rate. One can alternatively try to find a diagonal scaling matrix that maximize the rate bound (7). Trying to optimize this rate directly is difficult as it is a nonconvex nonsmooth problem [17]. Instead, we focus on optimizing ratio of the largest eigenvalue of Laplacian $\lambda_{\max}(\hat{\mathbf{L}_{\mathcal{G}}})$ and the smallest nonzero eigenvalue $\lambda_{\min>0}(\hat{\mathbf{L}_{\mathcal{G}}})$. Unfortunately, this is still untractable. To see this, notice that $\hat{\mathbf{L}_{\mathcal{G}}} = \hat{\mathbf{D}} - \hat{\mathbf{C}}\hat{\mathbf{E}}^{-1}\hat{\mathbf{C}}^\top = \hat{\mathbf{A}}^\top\mathbf{S}^\top\mathbf{S}\hat{\mathbf{A}} - \hat{\mathbf{A}}^\top\mathbf{S}^\top\mathbf{S}\hat{\mathbf{B}}(\hat{\mathbf{B}}\mathbf{S}^\top\mathbf{S}\hat{\mathbf{B}})^{-1}\hat{\mathbf{B}}\mathbf{S}^\top\mathbf{S}\hat{\mathbf{A}}$, which is nonconvex and nonsmooth in $\mathbf{S}$, and cannot be solved easily.

Instead of working on $\hat{\mathbf{L}_{\mathcal{G}}}$ directly, we can leverage the special structure of $\hat{\mathbf{L}_{\mathcal{G}}}$ to derive an upper bound and optimize this bound instead, which is easier to work with. We first define an auxliary matrix $\mathbf{G} = \mathbf{AA}^\top + \mathbf{BB}^\top$.

Lemma 1. *Matrix $\mathbf{G}$ is positive semidefinite, and its rank is*

$$\operatorname{rank}(\mathbf{G}) = \operatorname{rank}(\mathbf{E}) + \operatorname{rank}(\mathbf{L}_{\mathcal{G}}) = M + N - 1.$$

Lemma 1 shows that $\mathbf{G}$ is positive semidefinite with $M + N - 1$ nonzero eigenvalues, therefore we have $\lambda_{M+N-1}(\mathbf{G}) = \lambda_{\min>0}(\mathbf{G}) > 0$.

Theorem 1. *The eigenvalues of $\mathbf{L}_{\mathcal{G}}$ interlace with those of $\mathbf{G}$, that is,*

$$\lambda_i(\mathbf{G}) \geq \lambda_i(\mathbf{L}_{\mathcal{G}}) \geq \lambda_{i+M}(\mathbf{G}), \quad i = 1, 2, \ldots, N-1.$$

The eigenvalues of $\mathbf{L}_{\mathcal{G}} \in \mathbb{R}^{N\times N}$ are upper bounded by $\mathbf{G} \in \mathbb{R}^{T\times T}$. Together with Lemma 1, we can relate largest and smallest nonzero eigenvalues of $\mathbf{L}_{\mathcal{G}}$ and $\mathbf{G}$.

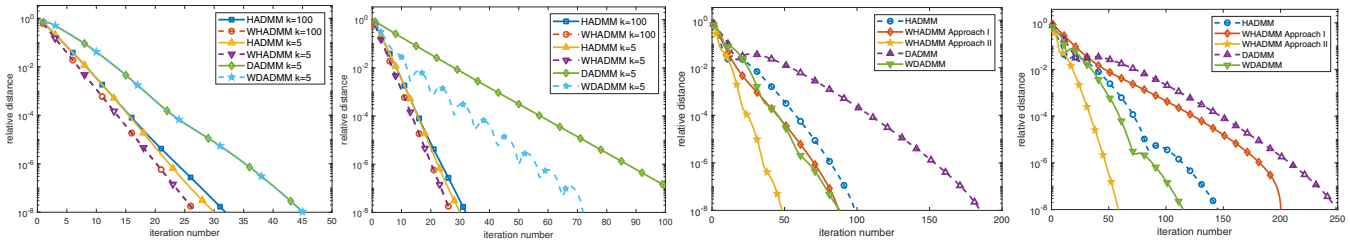
Corollary 1. *The largest and smallest nonzero eigenvalues of $\mathbf{L}_{\mathcal{G}}$ are bounded by those of $\mathbf{G}$, i.e.,*

$$\lambda_{max}(\mathbf{G}) \geq \lambda_{max}(\mathbf{L}_{\mathcal{G}}) \geq \ldots \geq \lambda_{min>0}(\mathbf{L}_{\mathcal{G}}) \geq \lambda_{min>0}(\mathbf{G}).$$

A direct consequence of Corollary 1 is that the condition number of $\mathbf{L}_{\mathcal{G}}$ is upper bounded by that of $\mathbf{G}$

$$\kappa(\mathbf{G}) \geq \kappa(\mathbf{L}_{\mathcal{G}}). \tag{9}$$

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 06,2025 at 09:34:47 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 2: Performance comparison of ADMM-based decentralized optimization algorithms over different graphs.

As a result, we can minimize the $\kappa(\mathbf{G})$ by choosing the proper scaling matrix $\mathbf{S}$. What makes this approach particularly appealing is that the size of $\mathbf{G}$ exactly matches the size of $\mathbf{A}$ and $\mathbf{B}$, which becomes apparent upon realizing

$$\mathbf{S}\mathbf{G}\mathbf{S}^\top = \mathbf{S}\mathbf{A}(\mathbf{S}\mathbf{A})^\top + \mathbf{S}\mathbf{B}(\mathbf{S}\mathbf{B})^\top = \hat{\mathbf{A}}\hat{\mathbf{A}}^\top + \hat{\mathbf{B}}\hat{\mathbf{B}}^\top.$$

Therefore, we can find an proper $\mathbf{S}$ by minimizing the upper bound $\kappa(\mathbf{G}) = \lambda_{\max}(\mathbf{S}\mathbf{G}\mathbf{S}^\top)/\lambda_{\min>0}(\mathbf{S}\mathbf{G}\mathbf{S}^\top)$. Problem (1) is a generalized eigenvalue problem [17], and can be solved using off-the-shelves methods [16–19].

4. NUMERICAL EXPERIMENTS

We consider the weighted network average problem with local observation $\mathbf{b}_i \in \mathbb{R}^3$. Let $\mathbf{x}_i$ be the decision variable of node i. The local objective function is $\frac{q_i}{2}\|\mathbf{x}_i - \mathbf{b}_i\|^2$, where q_i is the weight, and the overall objective is $f(\mathbf{x}) = \sum_{i=1}^N f_i(\mathbf{x}_i) = \frac{1}{2}(\mathbf{x}-\mathbf{b})^\top\mathbf{Q}(\mathbf{x}-\mathbf{b})$ where $\mathbf{Q} = \mathrm{diag}(q_1,\ldots,q_N)\otimes\mathbf{I}_3$, where $\mathbf{x} = (\mathbf{x}_1^\top,\ldots,\mathbf{x}_N^\top)^\top$ and $\mathbf{b} = (\mathbf{b}_1^\top,\ldots,\mathbf{b}_N^\top)^\top$. Let $\mathbf{x}^\star$ be the optimal solution. For a certain iteration k, we measure the quality of current solution by its relative distance to the optimal solution $\|\mathbf{x}-\mathbf{x}^\star\|/\|\mathbf{x}^\star\|$. We choose the best γ using grid search. We compare the preconditioned hybrid ADMM (WHADMM), plian vanilla hybrid ADMM (HADMM), decentralized ADMM (DADMM) and weighted decentralized ADMM (WDADMM) [13].

We first consider the case when $\mathbf{A}$ has full row rank. This happens when the underlying graph satisfies the condition that a single node is connected to all others. Typical examples include the star graph and lollipop graph, the latter of which is essentially attaching one more node to a compete graph. We test all the algorithms over a star graph and a lollipop graph, both of which have the same number of nodes, and the results are shown in Fig. 2 (a) and (b), respectively.

In both cases, hybrid methods (HADMM and WHADMM) consistently outperforms their decentralized counterparts (DADMM and WDADMM), which clearly demonstrates the benefits of hybrid methods. The gap is much larger in the lollipop graph case. Interestingly, the lollipop has much more edges than a star graph with the same number of nodes, but yields worse performance for DADMM and WDADMM. This result implies that more edges does not always lead to faster convergence. Since the scaling matrix $\mathbf{S}$ can be solved exactly in this case, we expect to see the same performance regardless of condition numbers, which is corroborated by comparing Fig. 1(a) and (b).

Next we consider the case when $\mathbf{A}$ does not have full row rank. To better illustrate the importance of edge weights, we choose graphs with two clusters connected by a single path. This path is critical to keep the graph connected. To check the sensitivity to topology changes, we test over two graphs with path length 2 and 4 hops, and the results are shown in Fig 1(c)(d). Both graphs have the same number of nodes. The test results can be found in Fig. 1(c) and (d).

A glance of both figures reveals that hybrid methods outperform decentralized ones. Moreover, one can see that WDADMM performs much better than DADMM, even better than WHADMM with Approach I, which corroborates that properly designed weights can be crucial for ADMM. Comparing two figures, we see that preconditioning greatly improves the performance as well as its robustness to topological changes of the underlying graph. The fact that WDADMM works pretty well showcases the power of considering edge weights. HADMM without weight tuning performs comparably with WDADMM validates the merits of adopting hybrid communication graphs. Also notice that preconditioning approach I consistently yields a worse convergence speed, even worse than HADMM without preconditioning, showing that this is not a good heuristic.

5. CONCLUSIONS AND DISCUSSION

In this paper, we consider solving decentralized optimization over weighted graphs, and we propose a preconditioning technique for finding the edge weights which lead to the fastest convergence of ADMM. Our method is based on a unified linear convergence rate, which not only is equivalent to existing results for star graphs, but also works for other types of graphs. Thanks to the unifying framework of hybrid ADMM, our method naturally reduces to vanilla decentralized ADMM by properly construction communication graphs.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 06,2025 at 09:34:47 UTC from IEEE Xplore. Restrictions apply.

6. REFERENCES

[1] Michael Rabbat and Robert Nowak, "Distributed optimization in sensor networks," in *Proceedings of the Third International Symposium on Information Processing in Sensor Networks - IPSN'04*, Berkeley, California, USA, 2004, p. 20, ACM Press.

[2] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Mirection Method of Multipliers," *Found. Trends® Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2011.

[3] G. B. Giannakis, Q. Ling, G. Mateos, I. D. Schizas, and H. Zhu, "Decentralized Learning for Wireless Communications and Networking," in *Splitting Methods in Communication, Imaging, Science, and Engineering*, R. Glowinski, S. J. Osher, and W. Yin, Eds., pp. 461–497. Springer, Cham, 2016.

[4] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su, "Scaling Distributed Machine Learning with the Parameter Server.," in *OSDI*, 2014, vol. 14, pp. 583–598.

[5] A. Nedic and A. Ozdaglar, "Distributed Subgradient Methods for Multi-Agent Optimization," *IEEE Trans. Autom. Control*, vol. 54, no. 1, pp. 48–61, Jan. 2009.

[6] Angelia Nedic, Asuman Ozdaglar, and Pablo A. Parrilo, "Constrained consensus and optimization in multi-agent networks," *IEEE Trans. Autom. Control*, vol. 55, no. 4, pp. 922–938, 2010.

[7] S. Sundhar Ram, A. Nedić, and V. V. Veeravalli, "Distributed Stochastic Subgradient Projection Algorithms for Convex Optimization," *J Optim Theory Appl*, vol. 147, no. 3, pp. 516–545, Dec. 2010.

[8] Dimitri P. Bertsekas and John N. Tsitsiklis, *Parallel and Distributed Computation: Numerical Methods*, vol. 23, Prentice-Hall, Englewood Cliffs, NJ, 1989.

[9] I. D. Schizas, A. Ribeiro, and G. B. Giannakis, "Consensus in Ad Hoc WSNs With Noisy Links – Part I: Distributed Estimation of Deterministic Signals," *IEEE Trans. Signal Process.*, vol. 56, no. 1, pp. 350–364, Jan. 2008.

[10] Wei Shi, Qing Ling, Kun Yuan, Gang Wu, and Wotao Yin, "On the Linear Convergence of the ADMM in Decentralized Consensus Optimization," *IEEE Trans. Signal Process.*, vol. 62, no. 7, pp. 1750–1761, Apr. 2014.

[11] Meng Ma, Athanasios N. Nikolakopoulos, and Georgios B. Giannakis, "Hybrid ADMM: A unifying and fast approach to decentralized optimization," *EURASIP J. Adv. Signal Process.*, vol. 2018, no. 1, pp. 73, Dec. 2018.

[12] P. Giselsson and S. Boyd, "Linear Convergence and Metric Selection for Douglas-Rachford Splitting and ADMM," *IEEE Trans. Autom. Control*, vol. 62, no. 2, pp. 532–544, Feburary 2017.

[13] Q. Ling, Y. Liu, W. Shi, and Z. Tian, "Weighted ADMM for Fast Decentralized Network Optimization," *IEEE Trans. Signal Process.*, vol. 64, no. 22, pp. 5930–5942, Nov. 2016.

[14] Marianna Bolla, "Spectra, Euclidean representations and clusterings of hypergraphs," *Discrete Mathematics*, vol. 117, no. 1, pp. 19–39, July 1993.

[15] Meng Ma, Bingcong Li, and Georgios B. Giannakis, "Tight linear convergence rate of admm for decentralized optimization," *arXiv preprint arXiv:1905.10456*, 2019.

[16] Jeremy G. VanAntwerp and Richard D. Braatz, "A tutorial on linear and bilinear matrix inequalities," *Journal of Process Control*, vol. 10, no. 4, pp. 363–385, Aug. 2000.

[17] Stephen P. Boyd, Laurent El Ghaoui, Eric Feron, and Venkataramanan Balakrishnan, *Linear Matrix Inequalities in System and Control Theory*, Number vol. 15 in SIAM Studies in Applied Mathematics. SIAM, Philadelphia, PA, 1994.

[18] R. Braatz and M. Morari, "Minimizing the Euclidean Condition Number," *SIAM J. Control Optim.*, vol. 32, no. 6, pp. 1763–1768, Nov. 1994.

[19] Pontus Giselsson and Stephen Boyd, "Metric selection in fast dual forward–backward splitting," *Automatica*, vol. 62, pp. 1–10, Dec. 2015.

Authorized licensed use limited to: AMRITA VISHWA VIDYAPEETHAM AMRITA SCHOOL OF ENGINEERING. Downloaded on July 06,2025 at 09:34:47 UTC from IEEE Xplore. Restrictions apply.